

CSE 4102: Computer Graphics and Image Processing Laboratory

3D Target Shooter Game

By

Kazi Rifat Al Muin

Roll: 2107042



Submitted To

Md Tajmilur Rahman

Lecturer

Md. Mubtashim Abrar Nihal

Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
October, 2026

1 Project description and objectives

Abstract. 3D Target Shooter is a C++17/OpenGL 3.3 game that demonstrates how one centered cube can produce an entire interactive shooting arena. A shared transformation pipeline constructs walls, sheared supports, weapons, circular-looking targets, projectiles, people, birds and light fixtures. Seven Challenge levels introduce translation, multi-axis motion, rotation, cover and penalties. Three shading modes and day/night lighting make the consequences of graphics equations observable. This report emphasizes construction coordinates and measured examples from the actual game, rather than idealized replacement models.



(a) Completed arena and shooting lanes.



(b) The same world with the night rig.

Figure 1: The implemented environment. Gameplay captures show the complete presentation; isolated construction captures later use the same renderer and geometry.

Relationship to the proposed project

The original proposal calls for an arena, aiming gun, circular ring targets, stands, travelling bullets, environmental boxes, hit effects, lighting, multiple views and interactive control. All are represented in the final game. The implementation deliberately uses **only cube geometry**: barrels and bullets are cuboids, not analytic cylinders or spheres. A target’s circular silhouette is approximated by 32 thin cuboids; the rings are computed in its fragment shader. This distinction is essential when explaining what the submitted code actually renders.

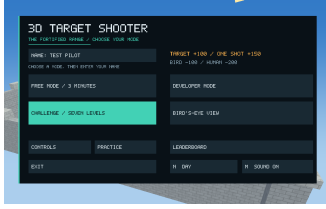
Objective	Implemented evidence
Construct meaningful objects	One shared 36-vertex cube; assemblies of independently transformed parts.
Demonstrate transformations	Scale, six shear terms, rotations around X/Y/Z, and world translation.
Animate and interact	Sinusoidal targets, aimed weapons, projectile travel, recoil, hit flash and fragments.
Explain illumination	Ambient, directional, eight point and six spot lights; diffuse/specular reflection.
Compare views and shading	Player, arena, side, free and Bird’s-Eye views; flat, Gouraud and Phong.
Make results reproducible	Fixed seeds and times; source-derived image guides and numerical traces.

Reading map and evidence

Physical pages 3–4 describe gameplay and levels; 5–14 explain geometry, assembly, motion and cameras; 15–18 cover textures, lighting and shading; 19–20 cover build, validation, limitations and references. The cover and references count toward the **20-page total**. Small panels are numbered construction steps, with matching coordinate tables. Units are meters, angles are degrees unless stated otherwise, and coordinates are rounded for readability. Full-precision, eight-corner traces remain in `objects/`; exhaustive lighting comparisons remain in `lighting/lighting.md`. Every selected report image is copied under `docs/images/report/` with provenance in `docs/report-image-manifest.csv`.

2 Gameplay, interaction and scoring

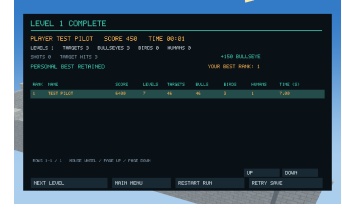
The player enters a callsign, selects a mode, aims at the printed front of a target and fires a suitable weapon. Challenge begins with a 1.5-second introduction; clearing all targets unlocks completion and the next level. After the seventh level the game shows victory/results. Menus, pause, introductions and result transitions suspend competitive simulation time. Focus loss pauses normal play. Nearby cover can obstruct the gun muzzle even when the camera can see a target.



(a) Mode selection.



(b) Aiming and compact HUD.



(c) Completed-level results.

Figure 2: The menu–play–results interaction cycle. The HUD contains score, progress, time, aim feedback, range and weapon controls.

Mode	Rules and persistence
Challenge	Seven sequential levels; accumulated score/time; eligible results after completed levels.
Free	Level-7 environment; 180-second session; targets return after 30 seconds; save at expiry.
Developer	Directly inspect any level; restart and diagnostics; no competitive record.
Bird’s-Eye	Level-7 overview and safe click-to-observe camera; noncompetitive, no shooting.
Practice	Seven sandbox targets; free movement; 1.8-second target respawn; separate scoring.

Hit rules. A target has 60 health. Ring index $r = 0, \dots, 5$ gives integer damage $D(r) = 60/(r + 1)$, hence 60, 30, 20, 15, 12, 10: respectively 1–6 equal-ring shots to destroy a fresh target. Back and edge contacts block a shot but do not award ring damage. All pellets from one trigger share a shot ID; the target records the *maximum* ring damage from that shot rather than summing pellets. Level modes award +150 for a one-shot target or +100 otherwise, with −100 per bird and −200 per human fatal hit. Practice adds applied damage plus a 100-point destruction bonus. Negative scores remain valid.

Input	Effect
WASD / Shift; mouse	Walk / sprint; adjust yaw and pitch.
Click or Space; 1/2/3	Fire (hold for rifle); select pistol/shotgun/rifle.
F1/F2/F3/F4; Q/E	Player/arena/side/free camera; descend/ascend in free view.
F6/F7/F8; N; M	Flat/Gouraud/Phong; day/night; sound on/off.
Esc; Enter; R; Tab	Pause/back; confirm/advance; restart; release mouse pointer.
Bird’s-Eye click/wheel/B	Observe a valid ground point; zoom; return overhead.
F5	Request a calculation CSV snapshot.

Leaderboard rows are separated by mode and normalized player name; duplicate names require explicit confirmation. Saves preserve best records and occur at eligible completion boundaries. CSV parsing supports quoted names and negative scores. See `SessionController.cpp`, `ScoreSystem.cpp` and the persistence sources for the exact state and storage rules.

3 Seven levels and bounded target motion

Level configuration supplies base position $\mathbf{b}$, amplitude $\mathbf{a}$, signed frequency ω , phase ϕ and spin s . All levels use the same world and gameplay builders. Levels 5–7 extend the preceding configuration rather than implementing separate engines.

Level	Targets	Walking	New difficulty in the actual configuration
1	3	Locked	Static targets at $(-5, 2.4, -18)$, $(0, 2.7, -22)$, $(5, 2.4, -18)$.
2	3	Locked	Center target moves horizontally: base $(0, 3, -31)$, amplitude $(2.3, 0, 0)$.
3	4	Locked	Separate X, Y and Z motion; phase-offset trajectories.
4	6	Enabled	Cargo screens, longer distances, rotating targets and navigable firing positions.
5	8	Enabled	Compound trajectories; eight configured birds.
6	10	Enabled	Faster three-axis trajectories; sixteen configured birds.
7	12	Enabled	Final pair has faster motion/spin; four birds and three humans per target $(48/36)$.



(a) Level 1: static introduction.



(b) Level 4: cover and movement.



(c) Level 7: populated final arena.

Figure 3: Representative progression; the target counts sum to 46 across Challenge.

$$p_j(t) = b_j + a_j \sin(t\omega_j + \phi_j), \quad \theta_y(t) = \text{fmod}(ts, 360). \quad (1)$$

For configuration variation v , let $d = -1$ if v is odd, otherwise $d = 1$. The builder sets

$$\omega = (kd, 1.13k, 0.87kd), \quad \phi = (0.63v, 0.47v, 0.81v), \quad s = s_0d.$$

Frequencies are radians per second; spin is degrees per second. A zero amplitude fixes that coordinate. For Level 2's center target, $v = 0$, $k = 1.1$, so $x(1) = 2.3 \sin(1.1) \approx 2.0498$, $y = 3$, $z = -31$; motion is bounded by $x \in [-2.3, 2.3]$. Its time-zero center remains $(0, 3, -31)$. Nonzero phases in later levels mean initial positions need not equal the configured base.

World validation. Locked-player levels test 32 trajectory samples at 0.43-second intervals for visibility/range. Walkable levels flood-fill a 51×91 meter grid, test intermediate positions to avoid corner cutting, and require a reachable clear firing position near every target. Cargo layout may retry up to four seeds, incremented by 7919. These checks prevent scenery from making objectives unreachable; sampled checks are not a proof for every continuous trajectory point.

Lifecycle. Elimination completes Challenge/Developer objectives; Free instead schedules 30-second respawn and a warning during its last three seconds. Surviving NPCs are reassigned without changing identity or duplicating penalties. Restart restores the appropriate completed-level statistics. Source: `src/levels/Level*.cpp`, `LevelManager.cpp`, `LevelWorld.cpp`, `Challenge.cpp`.

4 The shared cube and transformation pipeline

Every world part starts from eight corners ($\pm 0.5, \pm 0.5, \pm 0.5$). Six faces are expanded to 12 triangles (36 vertices) with separate face normals. One VAO/VBO stores this mesh; every `SceneObject` supplies its transform, color, material and lighting parameters. Assemblies are overlapping draw calls, not Boolean mesh unions. The `parent` field identifies an assembly; its transform is not automatically multiplied in the renderer.

$$\mathbf{p}_w = M\mathbf{p}_l, \quad M = TR_zR_yR_xHS, \quad \mathbf{p}_c = PVM\mathbf{p}_l, \quad \mathbf{p}_{ndc} = \mathbf{p}_c \cdot xyz / p_c \cdot w. \quad (2)$$

Column vectors are used, and column-major matrix storage is uploaded using `GL_FALSE`. Thus scale is applied first and translation last. With $c = \cos \theta$, $s = \sin \theta$ and $\theta = \pi \theta_{deg} / 180$:

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & h_{xy} & h_{xz} & 0 \\ h_{yx} & 1 & h_{yz} & 0 \\ h_{zx} & h_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

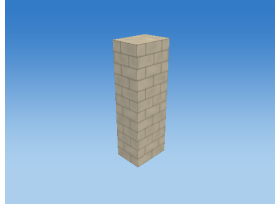
$$R_x^{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix}, \quad R_y^{3 \times 3} = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}, \quad R_z^{3 \times 3} = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Each rotation is extended with the homogeneous fourth row/column. A point has $w = 1$; a direction/offset has $w = 0$ so translation does not affect it. Each stage computes $p'_r = \sum_{c=0}^3 A_{rc} p_c$. The guides trace all eight corners through all six operations, including identities. **Exact composition loop from `Transform.h`:**

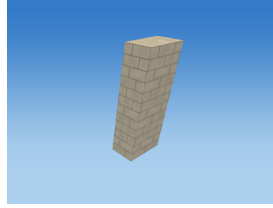
```
Mat4 model = Mat4::identity();
for (const auto& stage : modelTransformStages(t))
    model = stage.matrix * model;
return model;
```



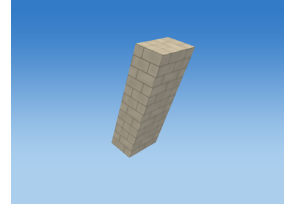
1. Unit cube.



2. Nonuniform scale.



3. XY shear.



4. Rotated and translated.

Figure 4: Shared transformation pipeline applied to a real boundary support. Intermediate stages are instructional shapes; the last panel is the unmodified game object.

Normal transformation. Nonuniform scale/shear require $\mathbf{N} = \text{normalize}((A^{-1})^T \mathbf{n}_l)$, $A = M_{3 \times 3}$. For columns $\mathbf{a}, \mathbf{b}, \mathbf{c}$, $\det A = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ and the normal-matrix columns are $(\mathbf{b} \times \mathbf{c}, \mathbf{c} \times \mathbf{a}, \mathbf{a} \times \mathbf{b}) / \det A$. Using $M\mathbf{n}_l$ directly would tilt lighting incorrectly under shear. The GPU then rasterizes triangles and interpolates attributes according to the selected shading mode [2].

5 Arena, walls and environmental construction

The rectangular floor is 60×100 m. Its center is $(0, -0.1, -50)$ and scale $(60, 0.2, 100)$, so its top lies at $y = 0$. The playable direction is $-Z$. Four continuous 8 m walls enclose the arena; supports and crenellations decorate rather than open the boundary.

Part	Center (x, y, z)	Scale (x, y, z)	Yaw
Floor	$(0, -.1, -50)$	$(60, .2, 100)$	0°
North / South wall	$(0, 4, -100)/(0, 4, 0)$	$(60, 8, 1)$	0°
West / East wall	$(-30, 4, -50)/(30, 4, -50)$	$(100, 8, 1)$	90°
Corner tower	$(\pm 30, 4.75, -100 \text{ or } 0)$	$(3.5, 9.5, 3.5)$	0°
Corner cap	$(\pm 30, 10, -100 \text{ or } 0)$	$(4.2, 1, 4.2)$	0°
End top blocks	$(x, 8.75, -100 \text{ or } 0)$	$(2, 1.5, 1.5)$	0°
Side top blocks	$(\pm 30, 8.75, z)$	$(2, 1.5, 1.5)$	90°

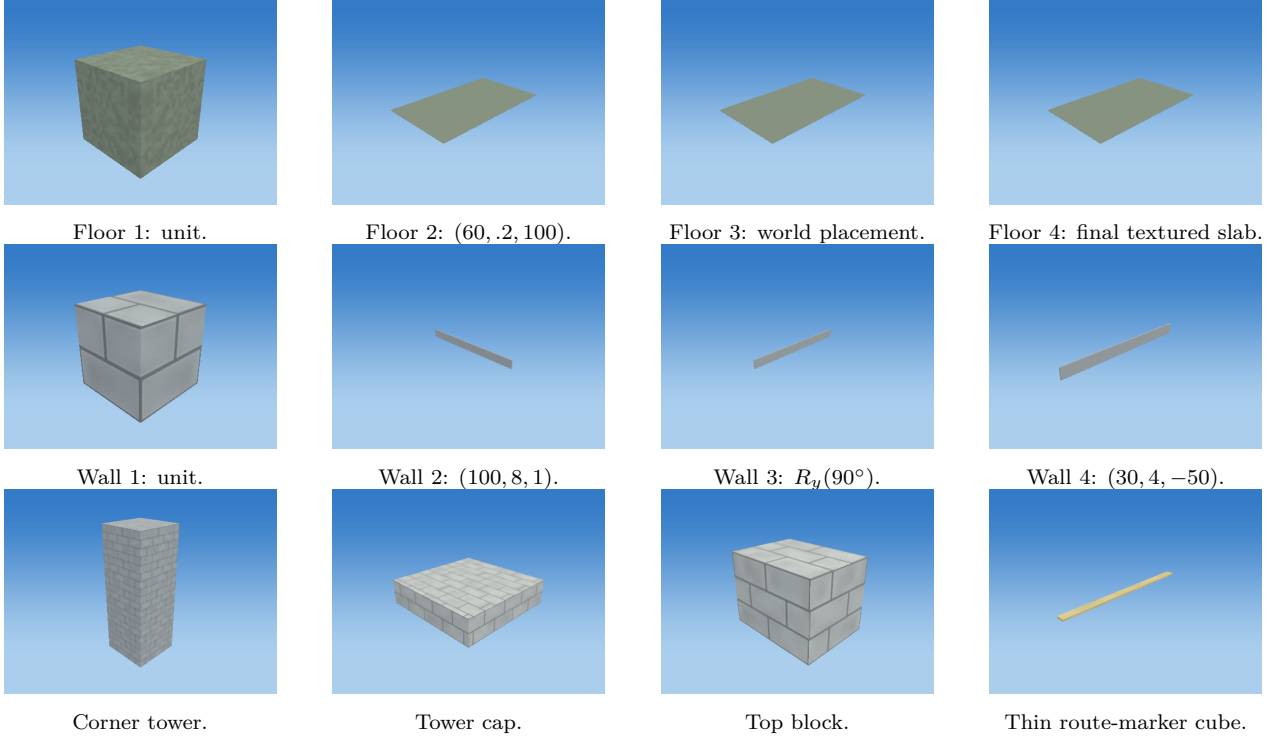


Figure 5: Arena parts are constructed from the shared cube. Final close views are reframed around world coordinates; camera reframing is not an extra model transformation.

For the east wall, $(.5, .5, .5) \xrightarrow{S} (50, 4, .5) \xrightarrow{R_y(90)} (.5, 4, -50) \xrightarrow{T} (30.5, 8, -100)$. The geometry spans $z = -100$ to 0 . Route markers use centers $(\pm 2.8, .012, z)$, $z = -8, -12, \dots, -92$, scale $(.075, .018, 1.35)$. Thin masonry string courses sit at heights 2, 4 and 6. The equipment table at $(-9, .55, -8)$ has scale $(12, 1.1, 3)$; its mat and three identification plates sit above it. The displayed weapons use the same builders as the held weapons. Source: `Arena.cpp` and `Environment.cpp`.

6 Worked shear, rotation and ground anchoring

The eastern support `SUPPORT_E_18` demonstrates a nonidentity shear and Z rotation. Its scale is $(1.5, 6, 2)$, $h_{xy} = 0.22$, rotation $(0, 0, -8^\circ)$ and translation $(28, 2.98332977, -18)$. The builder calculates the Y translation from the transformed corners, so the lowest point touches the ground.

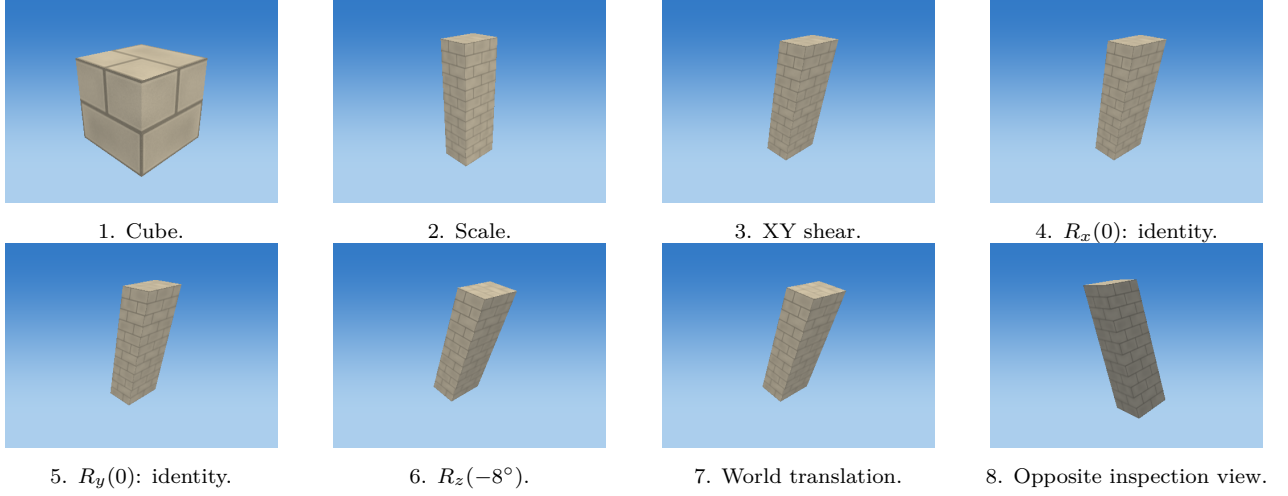


Figure 6: All transformation stages of the same support, including identity operations. Steps 2–6 share a camera; the unit and world views are independently framed.

Operation	Calculation for local corner $(.5, .5, .5)$	Result
Scale	$(1.5(.5), 6(.5), 2(.5))$	$(.75, 3, 1)$
Shear	$x \leftarrow .75 + .22(3)$	$(1.41, 3, 1)$
X and Y rotations	Both angles are zero	$(1.41, 3, 1)$
Z rotation	$x = c(1.41) - s(3), y = s(1.41) + c(3)$	$(1.8137973, 2.7745701, 1)$
Translation	Add $(28, 2.9833298, -18)$	$(29.8137973, 5.7578999, -17)$

Here $c = \cos(-8^\circ) = 0.9902681$ and $s = -0.1391731$. The resulting matrix, rounded to seven decimals, is

$$M = \begin{bmatrix} 1.4854021 & 2.1421925 & 0 & 28 \\ -.2087597 & 5.7578998 & 0 & 2.9833298 \\ 0 & 0 & 2 & -18 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The vertical extent is $e_y = \frac{1}{2}(|M_{10}| + |M_{11}| + |M_{12}|) \approx 2.9833298$, so setting $t_y = e_y$ makes $y_{min} = t_y - e_y = 0$. The actual code evaluates all eight corners before translating; both calculations describe the same affine box. Western supports use $(0, 180, 8)^\circ$; north/south supports use $(-8, 90, 0)^\circ$ and $(8, -90, 0)^\circ$. They retain the same scale/shear but are anchored after their own rotations. Collision uses this same composed matrix, including shear.

7 Cargo crates: deterministic multipart assembly

The reference human height is 2.1 m; crate side length is $s = 2.1/3 = 0.7$ m. One crate comprises a wood body, two metal bands and a paper inventory plate. The representative is the first Practice crate from seed 2107042. Its yaw is zero and body center is $\mathbf{b} = (-21.23, .35, -20)$. These are actual generated values, not a newly invented layout.

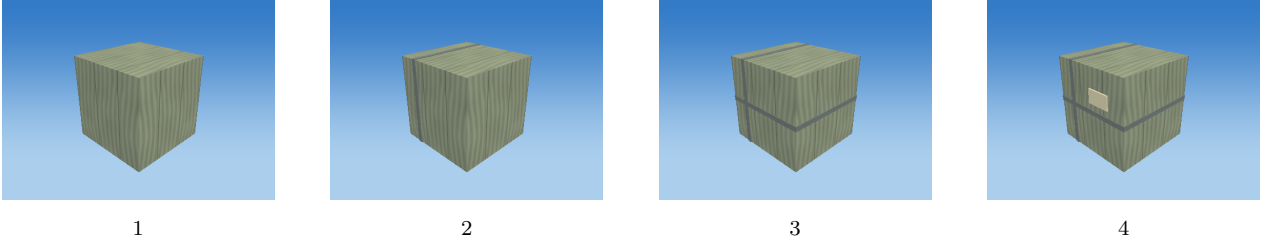


Figure 7: Cargo assembly in builder order: (1) body, (2) vertical band, (3) horizontal band, (4) inventory plate.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	Crate	$(-21.23, 0.35, -20)$	$(0.7, 0.7, 0.7)$	$(0, 0, 0)$
2	Crate band	$(-21.426, 0.35, -20)$	$(0.0385, 0.708, 0.708)$	$(0, 0, 0)$
3	Crate band	$(-21.23, 0.35, -20)$	$(0.708, 0.0385, 0.708)$	$(0, 0, 0)$
4	Crate inventory plate	$(-21.125, 0.483, -19.644)$	$(0.189, 0.112, 0.012)$	$(0, 0, 0)$

For every detail, $\mathbf{p} = \mathbf{b} + R_y(\theta)\mathbf{o}$ and its yaw is θ . The exact offset/size rules are

Part	Local offset $\mathbf{o}$	Size
Vertical band	$(-.28s, 0, 0) = (-.196, 0, 0)$	$(.055s, s + .008, s + .008)$
Horizontal band	$(0, 0, 0)$	$(s + .008, .055s, s + .008)$
Plate	$(.15s, .19s, .5s + .006)$	$(.27s, .16s, .012)$

Thus the plate center is $(-21.125, .483, -19.644)$ and its size is $(.189, .112, .012)$. The 0.008 and 0.006 offsets separate exposed surfaces and avoid placing the visible detail exactly coplanar with the crate face. The body bottom is $.35 - .7/2 = 0$.

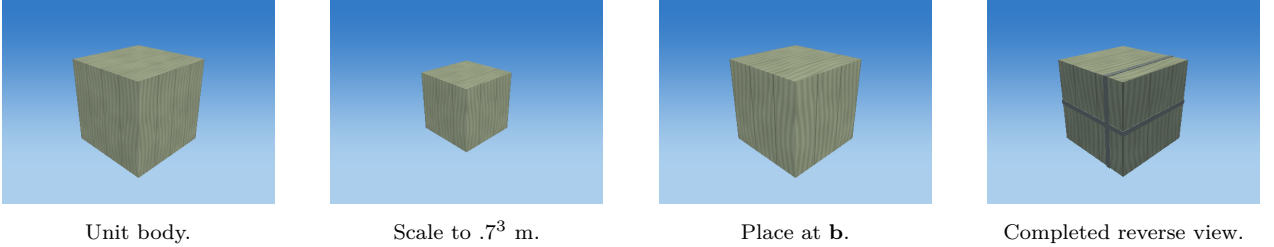


Figure 8: Body transformation and the opposite side of the final crate. All cubes retain independent transforms.

Repeatable layout. The generator uses `std::mt19937(seed)`; columns are spaced by $s + .035 = .735$ m. Side lanes are $x = -20, -15, 15, 20$ with jitter $(r \bmod 7 - 3) \cdot .08$. Row centers use $z = -20 - 10 \text{ row}$, runs have $3 + r \bmod 5$ columns, yaw is $90(r \bmod 4)$, and height is $1 + r_1 \bmod 3 + r_2 \bmod 3$ crates. L/T formations avoid duplicate joints. Higher levels add cover screens and remove conflicting crate assemblies using the target's swept amplitude plus clearance. Reserved central and side corridors keep movement possible. Source: `Cargo.cpp`, `Dimensions.h`; the generated guide preserves nine-significant-digit values.

8 Pistol: every cube and its coordinates

The pistol has 11 permanent cuboids. This fixed equipment-display example uses camera/anchor $\mathbf{c} = (-13, 1.65, -8)$, yaw -90° , pitch 0° and recoil 0; hence its orientation is identity. Each panel adds exactly one component.

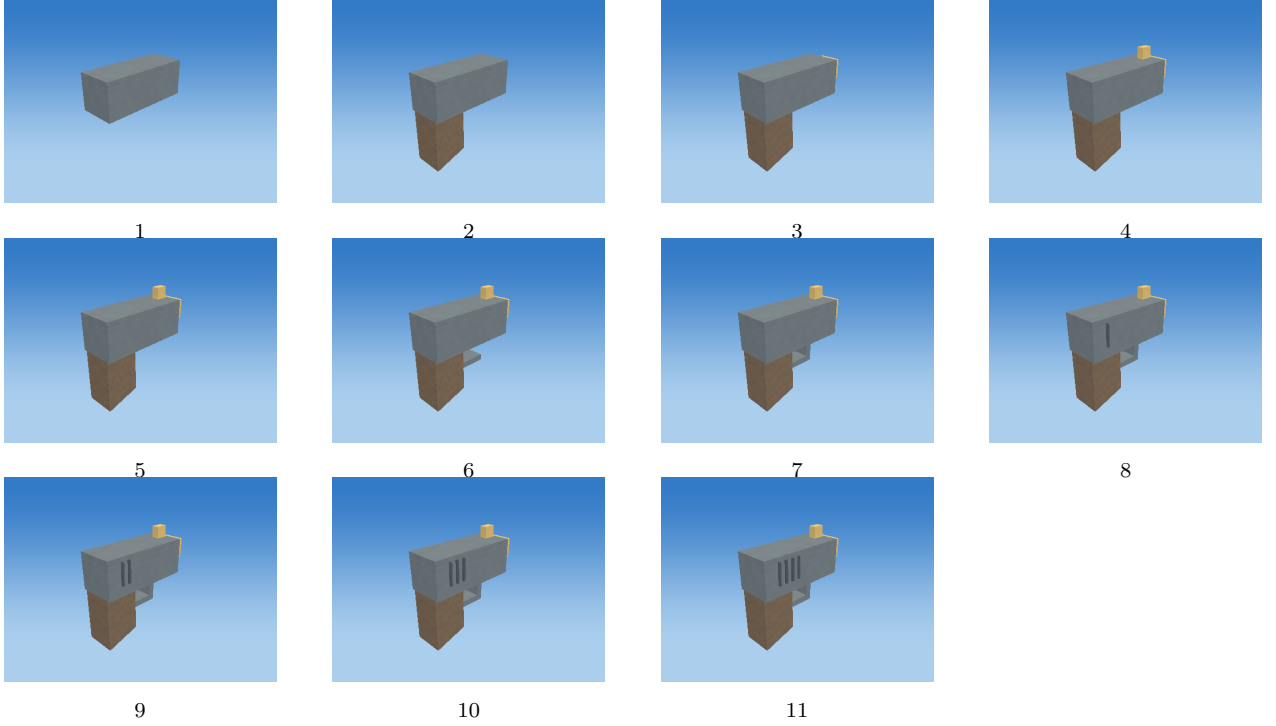


Figure 9: Pistol assembly: body, grip, barrel, sight, muzzle inset, lower trigger guard, front guard, then four side details. Panel numbers match the coordinate table.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	$(-12.68, 1.4, -8.68)$	$(0.22, 0.22, 0.56)$	$(0, 0, 0)$
2	GRIP	$(-12.68, 1.19, -8.5)$	$(0.18, 0.37, 0.2)$	$(0, 0, 0)$
3	BARREL	$(-12.68, 1.43, -8.96)$	$(0.13, 0.12, 0.16)$	$(0, 0, 0)$
4	SIGHT	$(-12.68, 1.55, -8.89)$	$(0.045, 0.07, 0.07)$	$(0, 0, 0)$
5	MUZZLE_INSET	$(-12.68, 1.43, -9.043)$	$(0.085, 0.075, 0.012)$	$(0, 0, 0)$
6	TRIGGER_GUARD	$(-12.68, 1.17, -8.68)$	$(0.13, 0.035, 0.19)$	$(0, 0, 0)$
7	GUARD_FRONT	$(-12.68, 1.25, -8.76)$	$(0.13, 0.16, 0.035)$	$(0, 0, 0)$
8	DETAIL_0	$(-12.567, 1.42, -8.49)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
9	DETAIL_1	$(-12.567, 1.42, -8.535)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
10	DETAIL_2	$(-12.567, 1.42, -8.58)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$
11	DETAIL_3	$(-12.567, 1.42, -8.625)$	$(0.015, 0.13, 0.013)$	$(0, 0, 0)$

Grip shear. Before rotation, $h_{yz} = -0.3$, so a scaled point becomes $(x, y - .3z, z)$. The grip's local corner $(.5, .5, .5)$ scales to $(.09, .185, .10)$, shears to $(.09, .155, .10)$, and translates by $(-12.68, 1.19, -8.5)$ to $(-12.59, 1.345, -8.4)$. Its matrix is

$$M_{grip} = \begin{bmatrix} .18 & 0 & 0 & -12.68 \\ 0 & .37 & -.06 & 1.19 \\ 0 & 0 & .20 & -8.50 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Aiming and recoil in gameplay. For player yaw ψ , pitch α , component offset $\mathbf{o}$ and recoil r ,

$$Q = R_y(-\psi - 90)R_x(\alpha), \quad \mathbf{p} = \mathbf{c} + Q(\mathbf{o} + (0, 0, .20r)), \quad M = T(\mathbf{p})R_y(-\psi - 90)R_x(\alpha)HS.$$

The held startup pose uses $\mathbf{c} = (0, 1.7, -5)$, $\psi = -90^\circ$, $\alpha = 3.8^\circ$. The body offset $(.32, -.25, -.68)$ becomes approximately $(.32, 1.49562, -5.69507)$. Every part follows the same orientation, so the whole gun aims coherently. Firing sets $r = 1$; it decays by $7\Delta t$. When $r > .65$, one extra emissive muzzle-flash cube is appended. Metal parts use specular $k_s = .75$ and exponent 80. Source: `Weapon.cpp`; these are cuboid barrels, not cylindrical meshes.

9 Shotgun and rifle: complete assembly sequences

Both long guns reuse the pistol's offset-to-world formula. The shotgun display anchor is $(-9, 1.65, -8)$; the rifle's is $(-5, 1.65, -8)$. Both have yaw -90° , pitch 0 and no recoil. The compact grids preserve every construction step; their coordinates below are extracted from the generated guides.

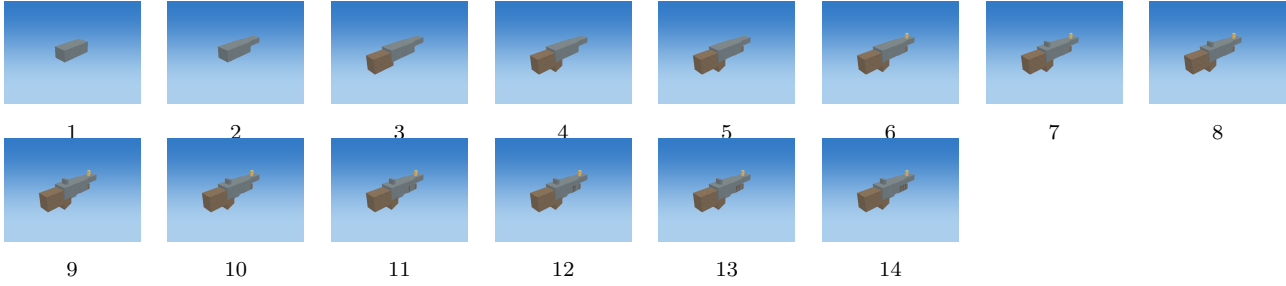


Figure 10: Shotgun: 14 cuboids, including the wider barrel, stock, fore-end, sights, trigger guard and four fore-end bands.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	$(-8.68, 1.37, -8.79)$	$(0.29, 0.25, 0.7)$	$(0, 0, 0)$
2	BARREL	$(-8.68, 1.43, -9.31)$	$(0.21, 0.13, 0.43)$	$(0, 0, 0)$
3	STOCK	$(-8.68, 1.29, -8.28)$	$(0.24, 0.3, 0.45)$	$(0, 0, 0)$
4	GRIP	$(-8.68, 1.15, -8.58)$	$(0.17, 0.27, 0.2)$	$(0, 0, 0)$
5	FORE_END	$(-8.68, 1.3, -9.05)$	$(0.28, 0.13, 0.34)$	$(0, 0, 0)$
6	FRONT_SIGHT	$(-8.68, 1.55, -9.34)$	$(0.05, 0.13, 0.07)$	$(0, 0, 0)$
7	REAR_SIGHT	$(-8.68, 1.55, -8.59)$	$(0.13, 0.1, 0.06)$	$(0, 0, 0)$
8	MUZZLE_INSET	$(-8.68, 1.43, -9.531)$	$(0.09, 0.075, 0.012)$	$(0, 0, 0)$
9	TRIGGER_GUARD	$(-8.68, 1.17, -8.76)$	$(0.13, 0.035, 0.19)$	$(0, 0, 0)$
10	GUARD_FRONT	$(-8.68, 1.25, -8.84)$	$(0.13, 0.16, 0.035)$	$(0, 0, 0)$
11	DETAIL_0	$(-8.68, 1.31, -8.95)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
12	DETAIL_1	$(-8.68, 1.31, -9.008)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
13	DETAIL_2	$(-8.68, 1.31, -9.066)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
14	DETAIL_3	$(-8.68, 1.31, -9.124)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$

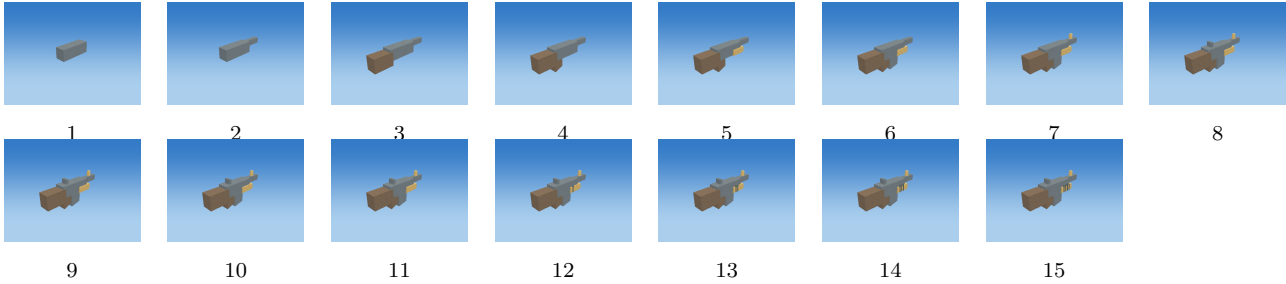


Figure 11: Rifle: 15 cuboids. The magazine is an additional sheared component; the narrower barrel and colored fore-end distinguish its silhouette.

Step	Component	World center (m)	Scale (m)	Rotation (deg)
1	BODY	$(-4.68, 1.37, -8.79)$	$(0.23, 0.25, 0.7)$	$(0, 0, 0)$
2	BARREL	$(-4.68, 1.43, -9.31)$	$(0.11, 0.13, 0.43)$	$(0, 0, 0)$
3	STOCK	$(-4.68, 1.29, -8.28)$	$(0.24, 0.3, 0.45)$	$(0, 0, 0)$
4	GRIP	$(-4.68, 1.15, -8.58)$	$(0.17, 0.27, 0.2)$	$(0, 0, 0)$
5	FORE_END	$(-4.68, 1.3, -9.05)$	$(0.28, 0.13, 0.34)$	$(0, 0, 0)$
6	MAGAZINE	$(-4.68, 1.14, -8.83)$	$(0.15, 0.35, 0.24)$	$(0, 0, 0)$
7	FRONT_SIGHT	$(-4.68, 1.55, -9.34)$	$(0.05, 0.13, 0.07)$	$(0, 0, 0)$
8	REAR_SIGHT	$(-4.68, 1.55, -8.59)$	$(0.13, 0.1, 0.06)$	$(0, 0, 0)$
9	MUZZLE_INSET	$(-4.68, 1.43, -9.531)$	$(0.09, 0.075, 0.012)$	$(0, 0, 0)$
10	TRIGGER_GUARD	$(-4.68, 1.17, -8.76)$	$(0.13, 0.035, 0.19)$	$(0, 0, 0)$
11	GUARD_FRONT	$(-4.68, 1.25, -8.84)$	$(0.13, 0.16, 0.035)$	$(0, 0, 0)$
12	DETAIL_0	$(-4.68, 1.31, -8.95)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
13	DETAIL_1	$(-4.68, 1.31, -9.008)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
14	DETAIL_2	$(-4.68, 1.31, -9.066)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$
15	DETAIL_3	$(-4.68, 1.31, -9.124)$	$(0.3, 0.15, 0.018)$	$(0, 0, 0)$

The long-gun grip has $h_{yz} = -.4$; the rifle magazine has $h_{yz} = -.22$. For the rifle magazine, scale $(.15, .35, .24)$ turns $(.5, .5, .5)$ into $(.075, .175, .12)$; shear makes $y = .175 - .22(.12) = .1486$. Adding its center $(-4.68, 1.14, -8.83)$ gives $(-4.605, 1.2886, -8.71)$. Shear changes the slant without introducing a different mesh. Shotgun fore-end uses Rubber, while the rifle's accent still follows the builder's material assignment. All part sizes, including the muzzle inset and detail strips, are shown above; the full guides additionally show their individual S/H/R/T images.

10 Circular targets and stands from 38 cubes

The representative Practice target is centered at $(0, 2.7, -19)$ with radius $r = .8$ m, thickness $.16$ m and yaw 0 . A base, pole, two braces and two warning strips form the first six cubes. The remaining 32 horizontal slices approximate the circular plate.



Figure 12: Every target assembly step. 1: base; 2: stand; 3/5: braces; 4/6: warning strips; 7–38: slices $i = 0 \dots 31$, added bottom to top. The rings are shader color, not extra concentric meshes.

$$h = 2r/32 = .05, \quad y_i = -r + (i + .5)h, \quad w_i = 2\sqrt{r^2 - y_i^2}, \quad (3)$$

$$\mathbf{p}_i = (0, 2.7 + y_i, -19), \quad S_i = (w_i, .05, .16), \quad R_i = R_y(\theta). \quad (4)$$

Slice	Local y_i	Width w_i	World center
0	-.775	.396863	$(0, 1.925, -19)$
7	-.425	1.355544	$(0, 2.275, -19)$
15	-.025	1.599219	$(0, 2.675, -19)$
16	.025	1.599219	$(0, 2.725, -19)$
31	.775	.396863	$(0, 3.475, -19)$

The stand height is $H = \max(.3, 2.7 - .8) = 1.9$. Base: center $(0, .15, -19)$, scale $(2, .3, 1.5)$. Pole: $(0, .95, -19)$, scale $(.24, 1.9, .24)$. For side $q = \pm 1$, brace center is $(.28q, .751, -19)$, scale $(.10, 1.064, .12)$, $R_z = 18q$ degrees; warning-strip center is $(.7q, .307, -19)$, scale $(.16, .012, 1.36)$.

Ring continuity across slices. Set $\text{patternScale} = (w_i/1.6, .05/1.6, 1)$ and $\text{patternOffset} = (0, y_i/1.6, 0)$. Then $\mathbf{q} = \mathbf{p}_i \odot \text{patternScale} + \text{patternOffset}$, $\rho = 2\|\mathbf{q}_{xy}\|$ and $\text{ring} = \text{clamp}(\lfloor 6\rho \rfloor, 0, 5)$. A dark separator appears when $\text{fract}(6\rho) > .95$. Only local +Z faces print the rings; all other faces use $(.15, .18, .21)$. Yaw rotates the slab thickness, face normals and scoring front together. Source: `Target.cpp`, `Target.h`, `object.frag`.

11 Projectiles, shooting and exact hit detection

All three projectile visuals use the shared cube with color $(1, .8, .22)$, emission $.6$ and specular $.8$. They are small elongated cuboids. They move along the actual muzzle-to-aim direction, not a fixed screen-space animation.

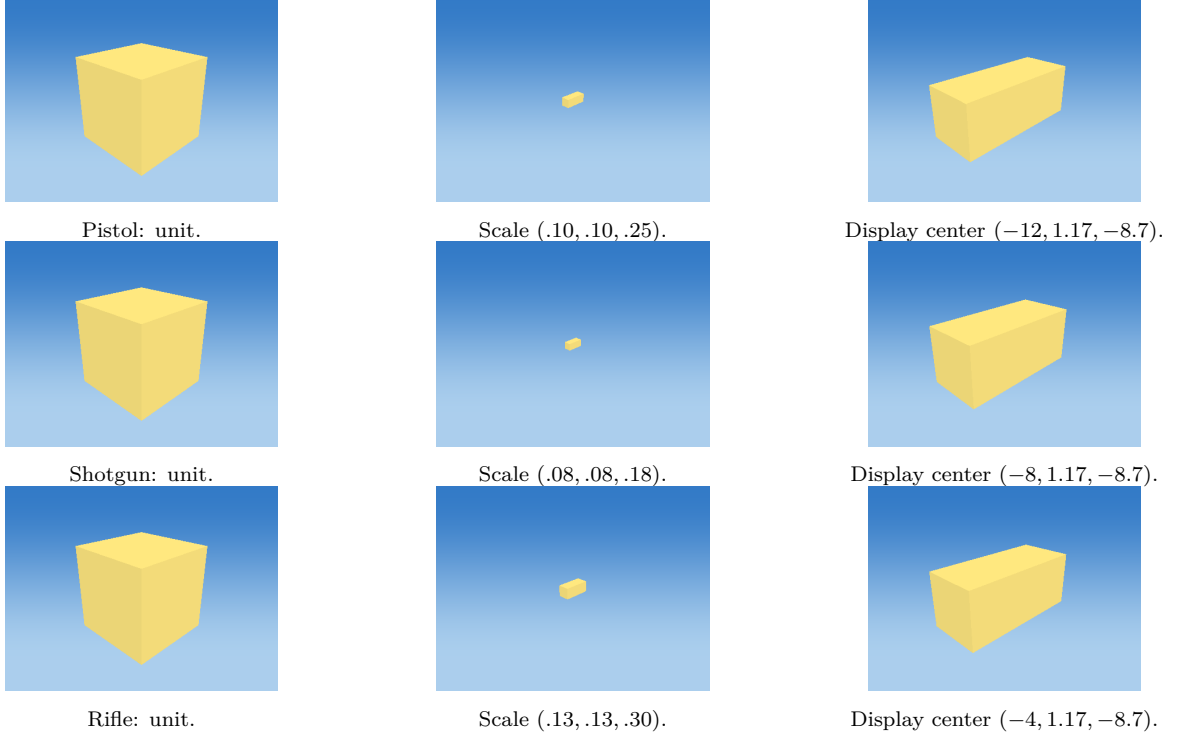


Figure 13: Deterministic equipment-display projectiles use the same builder and dimensions as fired projectiles. Their example direction is $(0, 0, -1)$, so their rotation is zero.

Weapon	Range (m)	Speed (m/s)	Cooldown (s)	Pellets	Spread
Pistol	25	42	.28	1	0
Shotgun	18	36	.80	9	.075
Rifle	70	65	.11	1	.009

The muzzle is $\mathbf{m} = \mathbf{c} + Q(.32, -.22, z_m)$, where $z_m = -1.02$ (pistol) or -1.52 (long guns). If the camera ray meets the nearest object at distance d_a , aim at $\mathbf{a} = \mathbf{c} + d_a \mathbf{f}$ and set $\mathbf{d} = \text{normalize}(\mathbf{a} - \mathbf{m})$. The shotgun emits one center pellet and eight with offsets $.075(\cos(2\pi i/8), \sin(2\pi i/8))$ in the ray's right/up basis. Rifle offsets are $.009(\sin(2.4n), \cos(1.7n))$ for shot count n .

Per update, travel is $\ell = \min(v\Delta t, \text{range} - \text{travelled})$, shortened to the nearest collision; $\mathbf{p} \leftarrow \mathbf{p} + \ell \mathbf{d}$. Simulation is subdivided to at most $1/120$ s. Visual angles are $\theta_x = \arcsin(d_y)$ and $\theta_y = \text{atan2}(-d_x, -d_z)$ in degrees.

Affine-box collision. Transform ray origin/direction by the inverse model basis: $\mathbf{o}_l = A^{-1}(\mathbf{o} - \mathbf{t})$, $\mathbf{d}_l = A^{-1}\mathbf{d}$ without renormalizing. Each axis intersects $[-.5, .5]$ using $t_1 = (-.5 - o_l)/d_l$, $t_2 = (.5 - o_l)/d_l$; the ray hits if the accumulated entry does not exceed exit. Parallel axes reject origins outside the slab. Targets test their actual slices after inverse yaw; NPCs test their animated cuboids. Front scoring additionally requires incoming local $d_z < 0$ and contact at $z = \text{thickness}/2$. This avoids treating the whole target's bounding square as its scoring circle.

12 People, birds and transient effects

NPCs share an offset transform: $\mathbf{p} = \mathbf{p}_{npc} + R_y(\text{heading})R_x(\text{fall})\mathbf{o}$, with component rotation $R_y(\text{heading})R_x(\text{fall})$. Humans are built from 21 cuboids; birds from 10. Their silhouettes and collision parts come from the same builders.

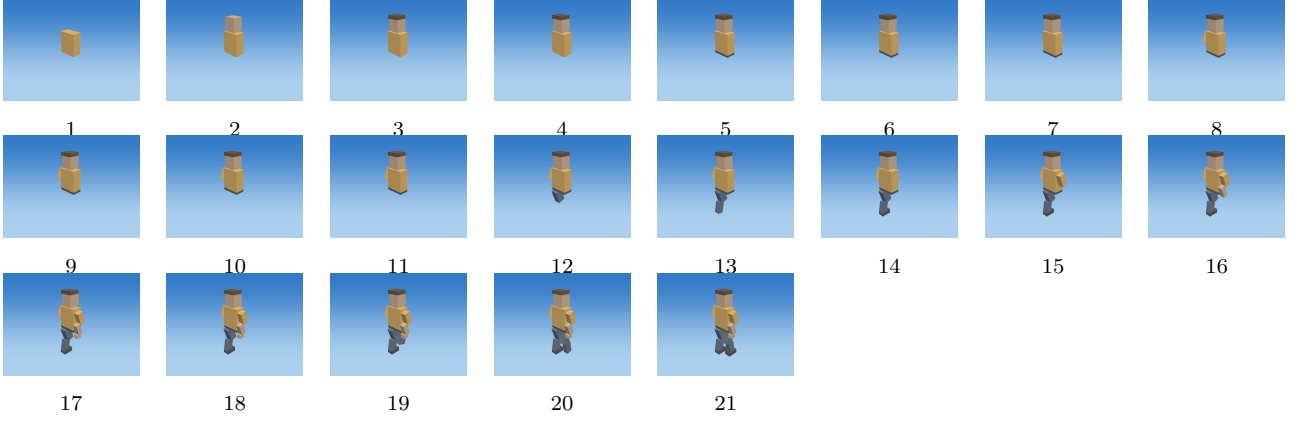


Figure 14: Human: torso, head, hair, neck, belt, vest stripe, nose, then seven parts for each side (upper arm, forearm, hand, eye, thigh, shin, shoe).

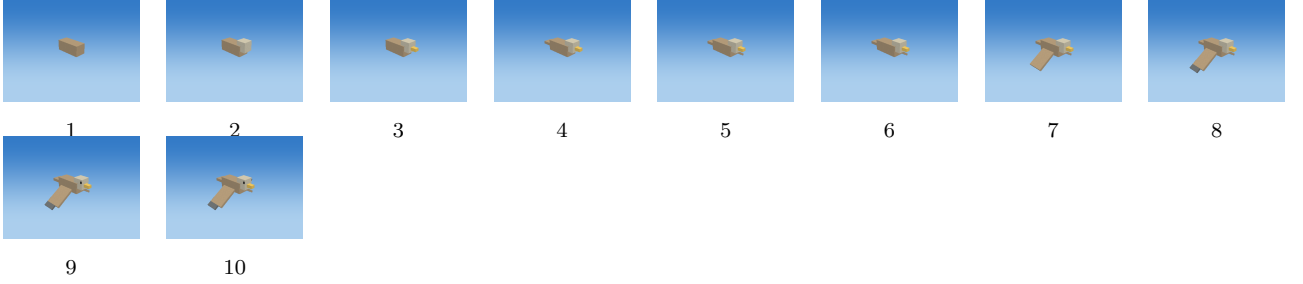


Figure 15: Bird: body, head, beak, tail, two wings and tips, then both eyes.

Human proportions and walking. Height is $H \in [2.05, 2.20]$ from the seeded generator; all offsets and sizes scale by $q = H/1.82$. Reference torso offset/size are $(0, 1.12, 0)$ and $(.46, .58, .25)$; head $(0, 1.61, 0)$ and $(.29, .34, .27)$; shoe $(\pm.14, .075, -.04)$ and $(.18, .12, .29)$. Thus at $H = 2.1$, $q = 1.153846$, torso size is $(.530769, .669231, .288462)$. This calculation illustrates the proportion formula; images retain their actual seeded height recorded in the human guide. Walk swing is $22 \sin(\text{animation})$ degrees, opposite on the two sides; forearm/shin offsets also use its sine. Human falling uses $90 \min(1, t/.75)^2$ degrees.

Bird articulation. Body size is $(.62, .26, .28)$; head center offset $(.32, .12 + b, 0)$, size $(.24, .24, .23)$; beak offset $(.49, .09 + b, 0)$, size $(.17, .07, .10)$. Bob $b = .045 \sin(\text{animation}/2)$; side- q flap $a_q = 35q \sin(\text{animation})$ degrees. Wing center is $(-.03, b - .35q \sin a_q, .35q \cos a_q)$ and scale $(.38, .055, .64)$. A fatal bird hit adds velocity, gravity 14 m/s^2 and pitch rate $260^\circ/\text{s}$ before settling.

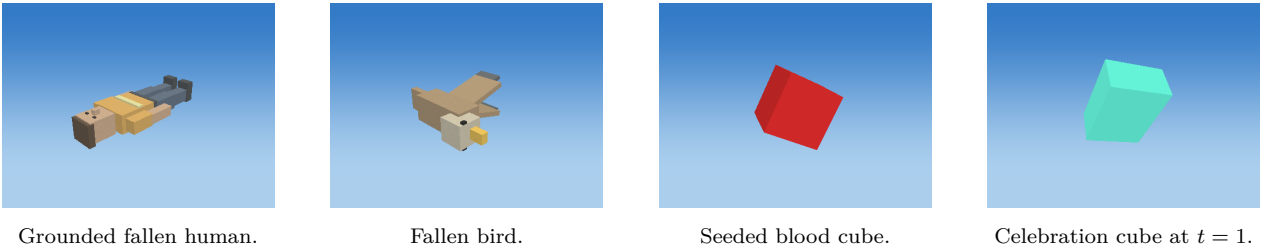


Figure 16: Temporary effects retain the same cube pipeline. They do not introduce new mesh primitives.

Effects equations. Target flash uses $f = \text{hitTime}/.25$ and blends the ring color toward $(1, .76, .2)$ by $f/2$. Breaking emits 12 fragments with $\alpha_i = 2\pi i/12$, $\mathbf{v}_i = (2.4 \cos \alpha_i, 2 + 2 \sin \alpha_i, 1.5)$ and lifetime $.75 \text{ s}$; gravity is 7. Blood uses gravity 13.5, seeded sizes $.035$ – $.07$, and flattens near the floor. For celebration particle i , age $a = t - .045(i \bmod 7)$, angle $\beta = 2.39996i$ and speed $u = 5 + 1.4(i \bmod 5)$ give $\mathbf{p} = (0, 22, -26) + (ua \cos \beta, (5 + i \bmod 4)a - 2.5a^2, ua \sin \beta)$. Body grounding uses the lowest transformed corner. Cosmetic particles expire and do not award additional score.

13 Camera views, projection and movement

The same world can be examined from the player, elevated arena, side and free cameras. Bird's-Eye adds an independent overhead/observation pair and visible ground marker. Inspection cameras do not reposition the actual grounded player or permit firing from an arbitrary flying position.

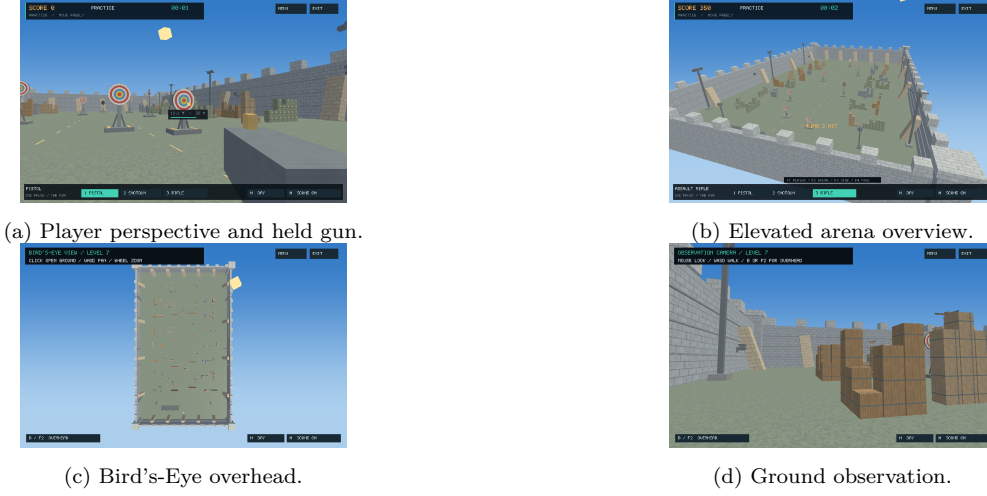


Figure 17: Camera changes alter the view matrix; model coordinates and game rules remain consistent.

Given yaw ψ and pitch α , the camera forward direction is

$$\mathbf{f} = \text{normalize}(\cos \psi \cos \alpha, \sin \alpha, \sin \psi \cos \alpha).$$

Mouse deltas give $\psi \leftarrow \psi + .12\Delta x$ and $\alpha \leftarrow \text{clamp}(\alpha - .12\Delta y, -89, 89)$. Let $\mathbf{s} = \text{normalize}(\mathbf{f} \times \mathbf{u}_0)$, $\mathbf{u} = \mathbf{s} \times \mathbf{f}$, $\mathbf{u}_0 = (0, 1, 0)$ and eye $\mathbf{e}$. Then

$$V = \begin{bmatrix} s_x & s_y & s_z & -\mathbf{s} \cdot \mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u} \cdot \mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f} \cdot \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad P = \begin{bmatrix} k/a & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & (F+n)/(n-F) & 2Fn/(n-F) \\ 0 & 0 & -1 & 0 \end{bmatrix},$$

where $k = 1/\tan(FOV/2)$, aspect $a = w/h$, and n, F are near/far planes. Gameplay uses a 60° perspective; isolated demonstration images use 45° and automatic framing so small parts remain visible. These are perspective views, not orthographic projections.

Movement. Grounded walking flattens and normalizes the forward vector, combines WASD directions, then moves at 5 m/s or 9 m/s while sprinting. Collision resolves X/Z separately in steps at most .15 m; the player remains within $x \in [-26, 26]$, $z \in [-96, -4]$. Free-camera speeds are 12/30 m/s and height is at least .3 m.

Picking. The overhead camera starts at $(0, 120, -50)$, yaw -90° , pitch -89.9° . For normalized cursor (u, v) , ray $\mathbf{r} = \text{normalize}[\mathbf{f} + (2u - 1)a \tan 30^\circ \mathbf{s} + (1 - 2v) \tan 30^\circ \mathbf{u}]$. Its ground intersection is $\mathbf{e} - \mathbf{r}(e_y/r_y)$. The game rejects unsafe/occluded positions, then places observation eye 1.7 m above valid ground. The marker uses scale $(.9, .08, .9)$, center $(x, .045, z)$; its heading cube follows observation yaw/pitch.

14 Texture generation, mapping and sampling

Nine original grayscale reflectance maps are generated offline by `tools/generate_textures.cpp`; they are not photographs or per-frame procedural shaders. The shipped P6 PPM files contain 256×256 RGB pixels. For this report their pixels are converted losslessly to PNG. Runtime loads them once into a `GL_TEXTURE_2D_ARRAY` with `GL_RGB8` storage.

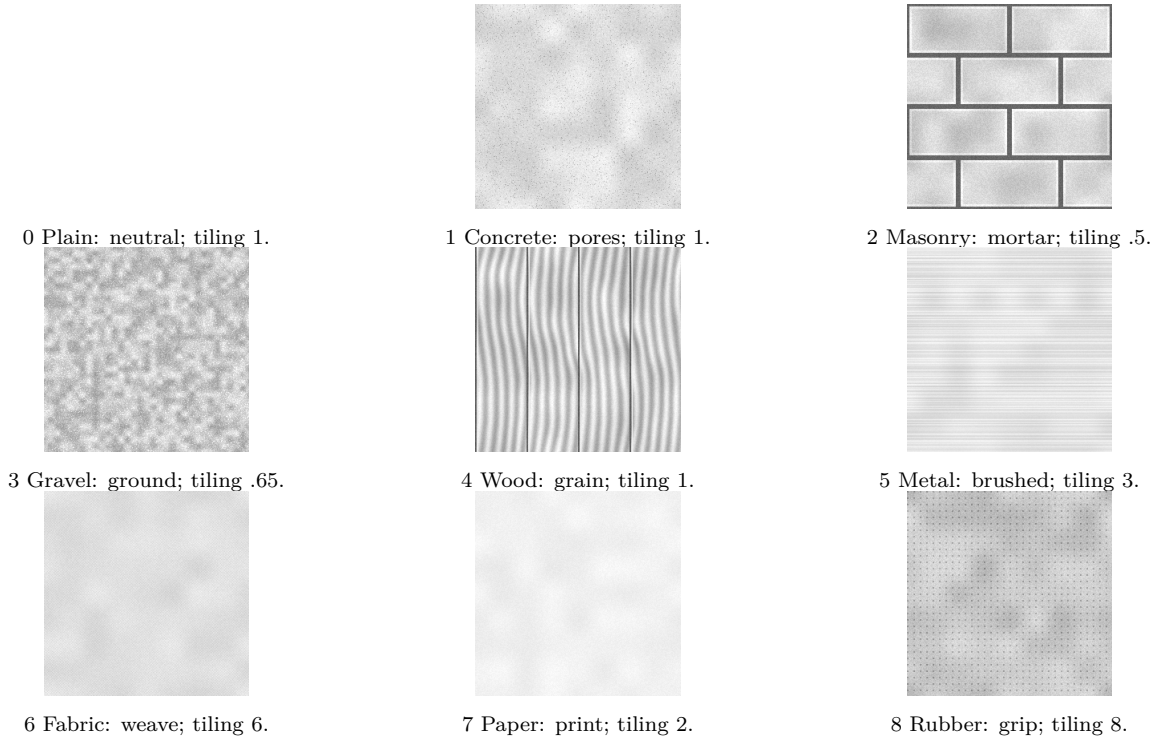


Figure 18: The actual texture pixels; tiling values are repeats per scaled local meter, not image enlargement factors.

Offline pattern formulas. Integer hash noise starts with $n = (x \& 255)374761393 + (y \& 255)668265263 + \text{seed}2246822519$ modulo 2^{32} , scrambles $n \leftarrow (n \oplus (n \gg 13))1274126177$, then returns $((n \oplus (n \gg 16)) \& 65535)/65535$. A wrapped grid interpolates noise using $g(t) = t^2(3 - 2t)$ in both dimensions. Masonry staggers 128-pixel bricks on 64-pixel courses; wood uses $\sin(.49x + 3\sin(2\pi y/256) + 2c)$ and plank seams every 64 pixels. Concrete/gravel combine coarse/fine noise; metal varies mainly along rows; fabric alternates $(x + y) \bmod 2$; rubber adds an 8-pixel grip grid. Values are clamped to $[0, 1]$, multiplied by 255 and copied to RGB.

Face-local UVs. For local position $\mathbf{p}_l$, scale $\mathbf{s}$ and tiling τ , let $\mathbf{q} = (\mathbf{p}_l + .5) \odot |\mathbf{s}| \tau$. Use (q_x, q_z) on Y faces, (q_z, q_y) on X faces and (q_x, q_y) on Z faces. A 60 m floor with $\tau = .65$ repeats 39 times across X and 65 times across Z. Because UVs derive from local position, rotation/translation do not cause texture swimming; shear stretches the pattern with the surface. This is planar face mapping, not blended triplanar mapping.

Sampling and color. Wrapping is `GL_REPEAT`; magnification is linear and minification is trilinear mipmap filtering. Mip levels are generated once. Base color is $\mathbf{B} = \mathbf{C}_{\text{surface}} \odot \text{texture}(\text{UV}, \text{layer})$ before lighting. Emission $> .5$ selects Plain; target-pattern surfaces select Paper; otherwise the assigned material is used. Maps are linear multipliers, with no automatic sRGB decoding. Nine base images occupy about 1.69 MiB; with mipmaps nominal RGB storage is about 2.25 MiB. There are no normal, roughness or displacement maps.

15 Ambient, directional and material lighting

The shader computes Phong reflection [4] from the actual world position and inverse-transpose normal. Let $\mathbf{N}$ be the normalized normal, $\mathbf{V} = \text{normalize}(\text{eye} - \text{position})$ and $\mathbf{L}$ point toward the light. A light's effective RGB intensity $\mathbf{C}$ already includes any attenuation/cone factor.

$$a = \max(\mathbf{N} \cdot \mathbf{L}, 0), \quad \mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}, \quad (5)$$

$$\mathbf{D} = \mathbf{A} + \sum_j \mathbf{C}_j a_j, \quad \mathbf{S} = \sum_{j: a_j > 0} \mathbf{C}_j k_s \max(\mathbf{R}_j \cdot \mathbf{V}, 0)^n. \quad (6)$$

This uses the reflection vector, not a halfway vector. Ambient adds a constant diffuse fill. Day ambient is $(.30, .32, .35)$; night is $(.012, .016, .025)$. Sun direction is $\text{normalize}(.45, .8, .3) \approx (.466002, .828449, .310668)$, with day color $(.95, .88, .73)$ and zero night intensity. It has no distance attenuation.



(a) Day ambient only, mask 1.



(b) Directional sun only, mask 2.



(c) Complete day preset, mask 15.



(d) Complete night preset, mask 15.

Figure 19: Actual shared-renderer captures using Phong shading and the same overview camera. Diagnostic masks remove source families; they do not invent additional gameplay presets.

For the upward floor normal $(0, 1, 0)$, the directional Lambert factor is $.828449$. Before texture/specular, day diffuse is approximately

$$(.30, .32, .35) + (.95, .88, .73)(.828449) = (1.087026, 1.049035, .954768).$$

Values may exceed one before the final clamp; there is no HDR tone-mapping operator. Default material values are $k_s = .12, n = 24$; weapons $.75, 80$; target plates/fixtures $.45, 48$; crate-body specular $.09$ and human $.06$.

$$\mathbf{L}_{lit} = \mathbf{B} \odot \mathbf{D} + \mathbf{S}, \quad \mathbf{C}_{out} = [\text{clamp}((1 - e)\mathbf{L}_{lit} + e\mathbf{B}, 0, 1)]^{1/2.2}.$$

Emission e blends toward unlit base color; it does not cast light onto nearby objects. The daytime sun is an emissive cube at $(25, 35, -80)$, scale $(4, 4, 4)$ and rotation $(0, 25, 15)^\circ$. It disappears at night; no moon is drawn. Lens emission is 1 at night and 0 by day. The sky is a separate fullscreen gradient $\text{mix}(\text{horizon}, \text{zenith}, \text{smoothstep}(.15, 1, uv_y))$, independent of scene lighting and camera orientation. **Shader connection.** The following statements are from `lighting.gls1`; attenuation is already included in `color` when this function is called.

```
float ndotl=max(dot(N,L),0.0);
diffuse+=color*ndotl;
vec3 R=reflect(-L,N);
```

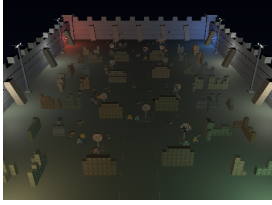
When `ndotl>0.0`, the next statement adds `color*materialSpecular*pow(max(dot(R,V),0.0),shininess)` to the specular accumulator. The explicit guard prevents highlights on a back-facing surface.

16 Point lamps, stadium spots and worked falloff

Night uses eight point lamps and six stadium spotlights. A visible fixture is geometry; the light position/direction is a separate uniform. Each stadium fixture has 12 cuboids and three lenses but only *one* spotlight. Day point/spot colors are zero.

$$\delta = \mathbf{p}_{light} - \mathbf{p}, \quad d = \max(\|\delta\|, .001), \quad \mathbf{L} = \delta/d, \quad A_p = \frac{1}{1 + .09d + .032d^2}, \quad A_s = \frac{1}{1 + .025d + .002d^2}.$$

For a spot, $c = \text{dot}(-\mathbf{L}, \mathbf{d}_{spot})$, $t = \text{clamp}((c - \cos 53^\circ)/(\cos 32^\circ - \cos 53^\circ), 0, 1)$ and cone intensity $I = t^2(3 - 2t)$. Thus the inner cone is fully lit, the penumbra is smooth and the outer region receives zero from that spotlight. Other lights may still illuminate it.



(a) Point family only.



(b) Spot family only.



(c) Night ambient+points+spots.



(d) Floor offset 1 m.



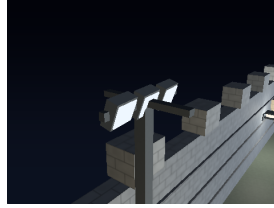
(e) Floor offset 5 m.



(f) Floor offset 12 m.



(g) Floor view on spot axis.



(h) Spot 0: three-lens fixture.



(i) Point 0: warm lamp fixture.

Figure 20: Source isolation, distance conditions and the corresponding physical fixtures. Close views move the camera, not lights or geometry. The cone equation controls illumination independently of the visible lens cubes.

Point example. Directly under the warm lamp $(-18, 4.7, -12)$, floor point $(-18, 0, -12)$ has $d = 4.7$, $\mathbf{L} = (0, 1, 0)$ and $A_p = 1/(1 + .423 + .70688) = .469510$ approximately. Its diffuse increment is $(2.2, 1.65, .9)A_p \approx (1.032922, .774692, .422559)$. This is one light's contribution before texture, other sources, emission and gamma, not a claimed measured framebuffer pixel.

Point positions are $(\pm 18, 4.7, -12)$, $(\pm 28.8, 5, -38)$, $(\pm 28.8, 5, -68)$ and $(\pm 28.8, 3.2, -89)$. Colors in source order are $(2.2, 1.65, .9)$, $(1.2, 2, 1.3)$, four $(1.7, 1.6, 1.4)$, then red $(2, .25, .15)$ and blue $(.25, .65, 2)$ accents. Spots sit at $(\pm 28, 12, z)$ for $z = -16, -48, -80$, point along $\text{normalize}(-x, -10, -4)$ and have color $(1.8, 1.9, 2.1)$.

Fixture construction uses pole center $(x, 6, z)$ and size $(.38, 12, .38)$; heads use rotation $(-\arcsin(d_y), \text{atan2}(d_x, d_z), 0)$ in degrees. Each lens is displaced $.18d$ from its head center. The general matrix and offset rules already demonstrated for weapons apply unchanged.

17 Flat, Gouraud and Phong shading compared

All modes use the same Phong *reflection equation*; the difference is where lighting is evaluated and how the result is interpolated [4, 5]. With separate cube face normals, Phong shading does not round the cube's geometric edges. Texture lookup and target-ring evaluation remain per fragment in every mode.

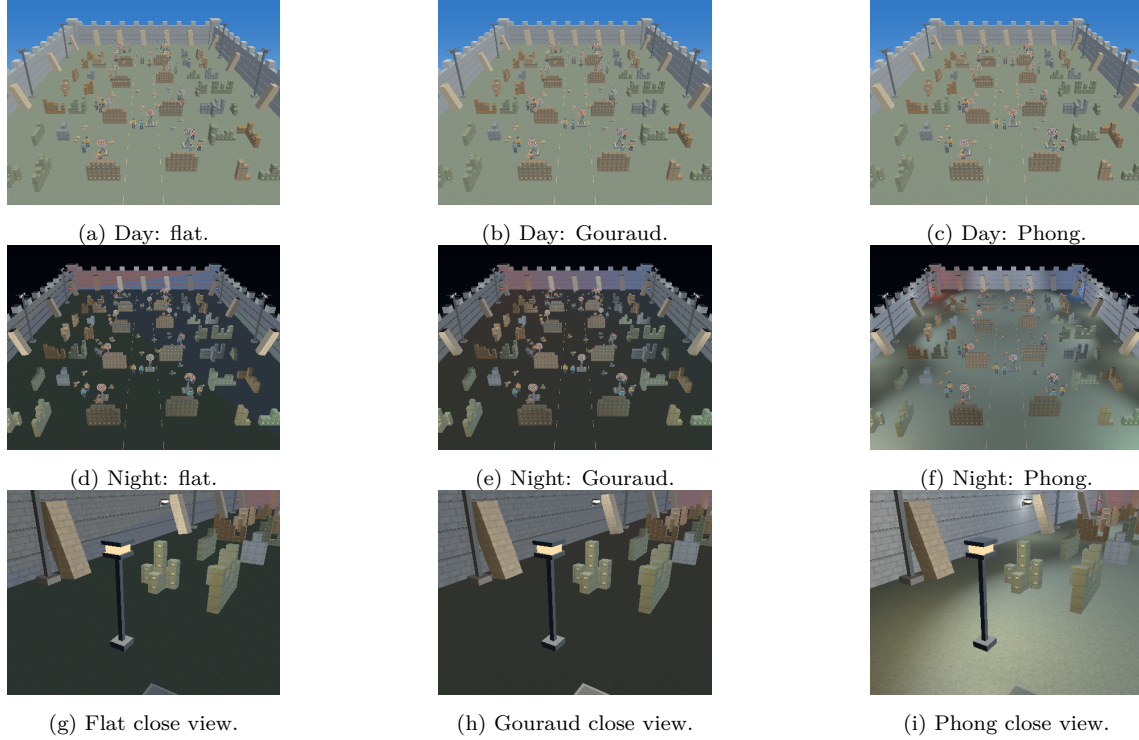


Figure 21: Controlled shading comparison: each row holds scene, camera and source mask fixed. Large floor/wall triangles expose the difference under local night lights.

Mode	Evaluation and interpolation	Consequence
Flat (F6)	Lighting evaluated at vertices; flat passes the provoking vertex result unchanged over the triangle.	A local source may produce a constant triangle-sized region.
Gouraud (F7)	Diffuse and specular computed at vertices and interpolated across the primitive.	Smooth vertex-to-vertex variation, but a highlight between vertices can be missed.
Phong (F8)	Interpolate position/normal; normalize and evaluate lighting in the fragment shader.	Distance, cone and highlights respond at fragment resolution; default mode.

For a smooth varying a , the rasterizer's perspective-correct interpolation has the form $a = (\sum_i \lambda_i a_i / w_i) / (\sum_i \lambda_i / w_i)$; flat varyings bypass this interpolation [2]. The implementation assigns both smooth and flat diffuse/specular outputs in `object.vert`; `object.frag` selects them for modes 0/1 or calls `lighting(worldPosition, worldNormal, ...)` for mode 2.

Coverage and limits. The lighting guide contains $2 \times 3 \times 16$ preset/shading/source-mask combinations, each at overview and close viewpoints: 192 comparison images. Mask bits are ambient=1, sun=2, points=4 and spots=8. Day masks differing only in point/spot bits are identical; night masks differing only in sun are identical. Emissive geometry and sky remain visible even with all four families removed. There are no cast shadows, shadow maps, ambient occlusion, bloom, normal mapping or physically based roughness. Dark areas reflect the stated lighting calculation, not a hidden shadow algorithm.

18 Software organization, build and verification

GLFW creates the window/context and supplies input/events; GLAD loads OpenGL entry points [3]. C++ scene/gameplay code owns model parameters, animation, collision and state. GLSL shaders handle vertex transformation, interpolation, texture sampling and lighting. The frame loop processes input, advances gameplay, builds the scene, draws world/sky and finally draws the UI. Audio is synthesized in code and played through Windows audio support; failure to open audio does not prevent gameplay.

Location	Responsibility
src/core/	Application loop, matrix helpers, scene data and affine collision.
src/world/, src/levels/	Arena, cargo, fixtures, textures' material choices and level configurations.
src/gameplay/	Weapons, targets, projectiles, NPCs, effects, scoring and session lifecycle.
src/rendering/, shaders/	Shared cube, uniform cache, texture array, object/sky/UI shaders.
src/persistence/, src/ui/	Leaderboards, calculation snapshots, names, menus, HUD and results.
objects/, lighting/	Deterministic demonstrations from the real builders and renderer.

Building/running. Use 64-bit MinGW, C++17 and an OpenGL 3.3-capable driver. Bundled GLFW is linked with `opengl32`, `gdi32` and `winmm`. From the repository root:

```
.\build.bat
.\main.exe
# Or: mingw32-make
# Packaged game: .\build\release\TargetShooter.exe
powershell -NoProfile -ExecutionPolicy Bypass -File tools/build_report.ps1
```

Keep `assets/` and `shaders/` next to the portable executable. The repository also contains `TargetShooter-share.zip`. An ordinary game build regenerates the demonstration guides, so it needs a working graphics context even if no source needs recompilation. Report preparation copies only selected images; it does not regenerate gameplay or overwrite source textures. PDF compilation uses two pdfLaTeX passes and checks the physical page count.

Practical rendering techniques. One cube mesh avoids repeated geometry upload. Uniform locations are cached; a numeric draw-state cache skips unchanged material uniforms. Day/night rigs are cached; zero-color lights are skipped. Texture images are uploaded once, with one array bind per world pass and numeric layer selection per object. CPU normal matrices avoid an inverse per vertex. Static scene vectors are reused; dynamic capacity is reserved. This is one draw per cube, not GPU instancing. Pending background CSV snapshots coalesce and flush on shutdown.

Verification evidence. Existing CPU suites cover matrix stages/six shears, collision, movement, ring damage, seed stability, level progression, NPC lifecycle and persistence. The previous demonstration build additionally checked all generated PNG decoding, SVG structure and Markdown references; inactive light-source masks were pixel-identical. A repeated unchanged game build regenerated byte-identical guides/images on the same GPU. The OpenGL smoke test exercised menus, views, all three weapons and day/night/shading. These are functional checks, not a claim of a particular frame rate on all hardware. The current report build separately checks copied-image hashes, unresolved references, layout warnings and the 20-page requirement.

The calculation CSV records actual current/visited objects rather than inventing unseen level observations. Leaderboard files are separate from documentation output. Reproducibility uses fixed scene seeds and explicit animation times; small GPU/driver rasterization differences across computers remain possible.

19 Conclusion, limitations and references

The project fulfills the target-shooting proposal through a consistent cube-based implementation. Its central graphics contribution is the ability to explain the final scene as a sequence of explicit scale, shear, rotation and translation operations, then connect those world coordinates to collision, camera projection, texture mapping and illumination. The report’s main examples preserve real builder values and show every assembly step for the three weapons, target, crate and NPC models within a fixed page budget.

The apparent circular target is a 32-slice approximation; cuboid projectiles/barrels and discontinuous face normals are deliberate implementation choices. There is no skeletal skinning, analytic cylinder/sphere mesh, Boolean merging, shadow pass or physically based material model. Lighting comparison images isolate finite implemented source families; they cannot exhaust every continuous camera position, moving-object pose or surface sample. Full generated guides contain the per-part, full-precision matrices and corner traces omitted from this compact report. Future extensions could add indexed curved meshes, instancing, shadow mapping and more scalable broad-phase collision while retaining one authoritative source for visuals and calculations.

Source and image attribution

All gameplay/construction/lighting captures and texture patterns are original project outputs by Kazi Rifat Al Muin. The source archive and README are the primary evidence for game-specific values [1]. The institution’s logo is used solely to identify the submitting institution, following the supplied cover template [6]. No unrelated sentiment-analysis content from the template is retained. The image manifest records original repository paths, copied report paths and SHA-256 hashes; converted texture PNGs preserve the source PPM pixels.

Report topic	Authoritative local implementation
Matrices / scene data	<code>src/core/Transform.h</code> , <code>src/core/SceneObject.h</code> .
Arena / cargo / fixtures	<code>src/world/Arena.cpp</code> , <code>Cargo.cpp</code> , <code>Lighting.cpp</code> .
Weapons / targets / shots	<code>src/gameplay/Weapon.cpp</code> , <code>Target.cpp</code> , <code>Projectile.cpp</code> , <code>Game.cpp</code> .
NPCs / cameras	<code>Bird.cpp</code> , <code>Human.cpp</code> , <code>Npc.cpp</code> , <code>src/camera/</code> .
Textures / shading	<code>TextureCache.cpp</code> , <code>Material.h</code> , <code>tools/generate_textures.cpp</code> , <code>shaders/</code> .
Detailed numerical evidence	<code>objects/README.md</code> , <code>lighting/lighting.md</code> , <code>docs/scene-reference.md</code> .

References

- [1] Kazi Rifat Al Muin, *3D Target Shooter: source code, README and generated object/lighting demonstrations*, project repository, 2026. <https://github.com/KaziRifatAlMuin/Graphics-Target-Shooter-Game>. Local source checkout is authoritative for the reported values.
- [2] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 3.3 (Core Profile)*, Khronos Group, March 11, 2010. <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf>. Reference for primitive processing, transformation, interpolation and texture operations.
- [3] GLFW, *Getting Started, GLFW 3.3 documentation*. <https://www.glfw.org/docs/3.3/quick.html>. Reference for window/context creation and event handling.
- [4] Bui Tuong Phong, “Illumination for Computer Generated Pictures,” *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, June 1975. <https://doi.org/10.1145/360825.360839>.
- [5] Henri Gouraud, “Continuous Shading of Curved Surfaces,” *IEEE Transactions on Computers*, vol. C-20, no. 6, pp. 623–629, June 1971. <https://doi.org/10.1109/T-C.1971.223313>.
- [6] Khulna University of Engineering & Technology, official institutional website. <https://www.kuet.ac.bd/>. Cover identity and institutional logo; cover layout follows `docs/template.tex`.